

Kiểm tra 3

1.

```
[6]: # Nguyễn Hoàng Tùng

# Import thư viện cần thiết
import pandas as pd
import os

# Đường dẫn thư mục chứa dữ liệu
paths = [
    r"D:\lecture\PTIT\20251\IntSys\Viet Nam Stock Market Data",
    r"D:\PTIT\20251\IntSys\Viet Nam Stock Market Data"
]

# Tên các file CSV
files = ["HOSEVNINDEXM30.csv", "HOSEVN100M30.csv", "HOSEVN30M30.csv"]

# Hàm tìm file trong các đường dẫn
def find_file(file_name, search_paths):
    for path in search_paths:
        full_path = os.path.join(path, file_name)
        if os.path.exists(full_path):
            return full_path
    return None

# Load các dataset
datasets = {}
for file in files:
    file_path = find_file(file, paths)
    if file_path:
        datasets[file] = pd.read_csv(file_path)
        print(f"{file} loaded successfully with shape {datasets[file].shape}")
    else:
        print(f"{file} not found in the given paths.")

# Kiểm tra dữ liệu đầu tiên
for name, df in datasets.items():
    print(f"\n{name} head:")
    display(df.head())
|
```

HOSEVNINDEXM30.csv loaded successfully with shape (22164, 9)

HOSEVN100M30.csv loaded successfully with shape (22158, 9)

HOSEVN30M30.csv loaded successfully with shape (21977, 9)

HOSEVNINDEXM30.csv head:

	time	open	high	low	close	55EMA	89EMA	200EMA	Plot
0	1451872800	579.03	579.46	578.33	579.28	NaN	NaN	NaN	NaN
1	1451874600	579.48	579.97	578.99	579.67	NaN	NaN	NaN	NaN
2	1451876400	579.55	579.55	578.14	578.52	NaN	NaN	NaN	NaN
3	1451878200	579.34	579.34	577.76	577.94	NaN	NaN	NaN	NaN
4	1451880000	577.84	579.32	577.44	579.32	NaN	NaN	NaN	NaN

HOSEVN100M30.csv head:

	time	open	high	low	close	55EMA	89EMA	200EMA	Plot
0	1451872800	573.52	573.52	571.10	572.00	NaN	NaN	NaN	NaN
1	1451874600	572.41	573.14	571.89	572.89	NaN	NaN	NaN	NaN
2	1451876400	572.61	572.61	571.20	571.42	NaN	NaN	NaN	NaN
3	1451878200	572.30	572.52	571.00	571.23	NaN	NaN	NaN	NaN
4	1451880000	571.22	572.72	570.69	572.72	NaN	NaN	NaN	NaN

HOSEVN30M30.csv head:

	time	open	high	low	close	55EMA	89EMA	200EMA	Plot
0	1451872800	595.57	595.57	592.70	593.48	NaN	NaN	NaN	NaN
1	1451874600	594.01	594.68	593.38	594.33	NaN	NaN	NaN	NaN
2	1451876400	594.01	594.01	592.39	592.83	NaN	NaN	NaN	NaN
3	1451878200	593.80	594.20	592.27	592.60	NaN	NaN	NaN	NaN
4	1451880000	592.52	594.08	591.83	594.08	NaN	NaN	NaN	NaN

2.

```
[8]: # Nguyễn Hoàng Tùng

# Import thư viện
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import matplotlib.pyplot as plt

# -----
# Chuẩn bị dữ liệu
# -----
dfs = []
for name, df in datasets.items():
    df = df.sort_values('time')
    df = df[['close']].copy()
    dfs.append(df)

# Kết hợp dữ liệu theo index
combined_data = pd.concat(dfs, axis=1)
combined_data.columns = ['close1', 'close2', 'close3']
combined_data.dropna(inplace=True)

# Chuẩn hóa dữ liệu
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(combined_data)

# -----
# Tạo dữ liệu chuỗi cho LSTM
# -----
def create_sequences(data, seq_length=5):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length, 0]) # Dự đoán close1
    return np.array(X), np.array(y)

SEQ_LENGTH = 10 # Sử dụng 10 ngày trước để dự đoán ngày tiếp theo
X, y = create_sequences(scaled_data, SEQ_LENGTH)

# Chia train/test (80%-20%)
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# -----
# Xây dựng mô hình LSTM >=5 Layer
# -----
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(SEQ_LENGTH, X_train.shape[2])),
    LSTM(64, return_sequences=True),
    LSTM(32, return_sequences=True),
    LSTM(16, return_sequences=True),
    LSTM(8),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()

# -----
# Huấn luyện mô hình
# -----
history = model.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.1)

# -----
# Đánh giá mô hình
# -----
y_pred = model.predict(X_test)

plt.figure(figsize=(12,6))
plt.plot(y_test, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.title('Dự đoán giá đóng cửa close1 bằng LSTM')
plt.legend()
plt.show()
```

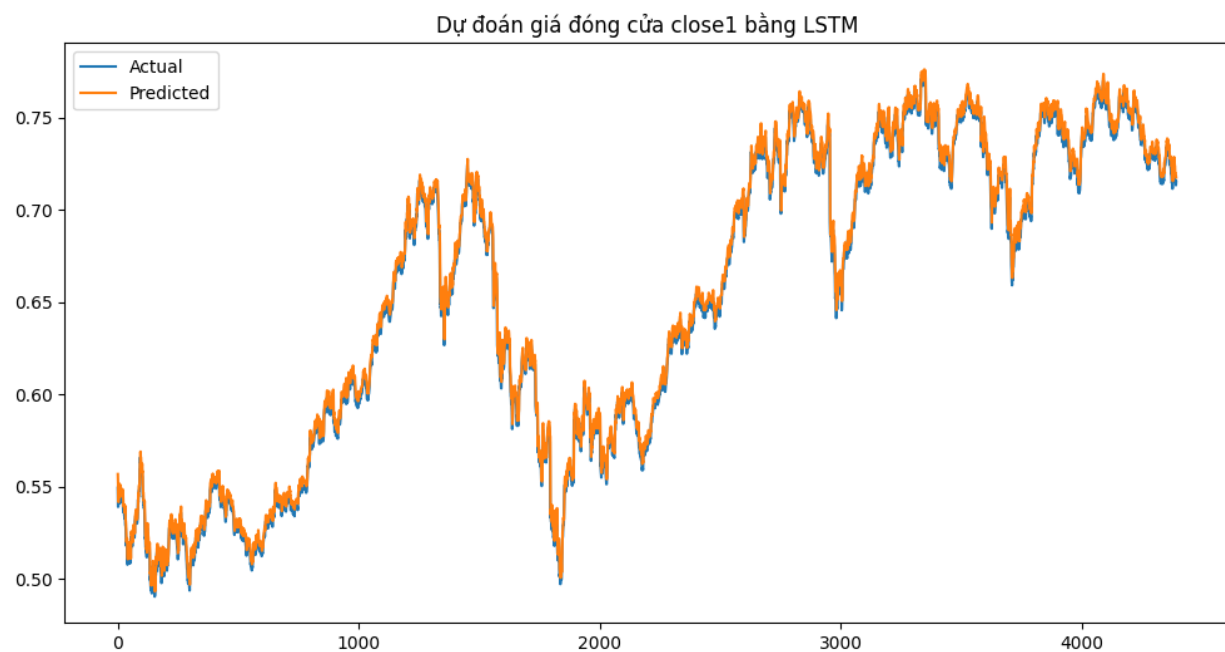
Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 10, 128)	67,584
lstm_1 (LSTM)	(None, 10, 64)	49,408
lstm_2 (LSTM)	(None, 10, 32)	12,416
lstm_3 (LSTM)	(None, 10, 16)	3,136
lstm_4 (LSTM)	(None, 8)	800
dense_6 (Dense)	(None, 1)	9

Total params: 133,353 (520.91 KB)

Trainable params: 133,353 (520.91 KB)

Non-trainable params: 0 (0.00 B)



3.

[9]: # Nguyễn Hoàng Tùng

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

# -----
# 1. Baseline: Naive Forecast
# -----
y_naive = scaled_data[SEQ_LENGTH-1:-1, 0] # Giá hôm nay làm dự đoán ngày mai
y_true = scaled_data[SEQ_LENGTH:, 0]
mse_naive = mean_squared_error(y_true, y_naive)
rmse_naive = np.sqrt(mse_naive)

# -----
# 2. Baseline: Moving Average (5 ngày)
# -----
window = 5
y_ma = []
for i in range(SEQ_LENGTH, len(scaled_data)):
    y_ma.append(np.mean(scaled_data[i-window:i, 0]))
y_ma = np.array(y_ma)
mse_ma = mean_squared_error(y_true, y_ma)
rmse_ma = np.sqrt(mse_ma)

# -----
# 3. Baseline: Linear Regression
# -----
# X_baseline = close 1,2,3 ngày hiện tại
X_lr = scaled_data[SEQ_LENGTH-1:-1, :] # hiện tại
y_lr = y_true
lr = LinearRegression()
lr.fit(X_lr, y_lr)
y_pred_lr = lr.predict(X_lr)
mse_lr = mean_squared_error(y_lr, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
```

```
# -----
# 4. Mô hình LSTM
# -----
y_pred_lstm = model.predict(X_test)
# X_test là scaled, y_test đã chuẩn hóa
rmse_lstm = np.sqrt(mean_squared_error(y_test, y_pred_lstm))

# -----
# 5. In kết quả
# -----
print(f"RMSE Naive Forecast: {rmse_naive:.5f}")
print(f"RMSE Moving Average (5 days): {rmse_ma:.5f}")
print(f"RMSE Linear Regression: {rmse_lr:.5f}")
print(f"RMSE LSTM: {rmse_lstm:.5f}")
```

138/138 ————— 1s 6ms/step
RMSE Naive Forecast: 0.00358
RMSE Moving Average (5 days): 0.00560
RMSE Linear Regression: 0.00356
RMSE LSTM: 0.00457

4.

```
[12]: # Nguyễn Hoàng Tùng

# -----
# Import thư viện
# -----
import pandas as pd
import numpy as np
import datetime
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# -----
# Chuẩn bị dữ liệu
# -----
dfs = []
for name, df in datasets.items():
    df = df.sort_values('time')
    df = df[['close']].copy()
    dfs.append(df)

combined_data = pd.concat(dfs, axis=1)
combined_data.columns = ['close1', 'close2', 'close3']
combined_data.dropna(inplace=True)

# Chuẩn hóa dữ liệu
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(combined_data)

SEQ_LENGTH = 10 # Sử dụng 10 ngày gần nhất

# -----
# Tạo chuỗi dữ liệu cho LSTM
# -----
def create_sequences(data, seq_length=10):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length]) # dự đoán toàn bộ vector [close1, close2, close3]
    return np.array(X), np.array(y)

X, y = create_sequences(scaled_data, SEQ_LENGTH)
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# -----
# Huấn Luyện mô hình LSTM
# -----
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(SEQ_LENGTH, X_train.shape[2])),
    LSTM(64, return_sequences=True),
    LSTM(32, return_sequences=True),
    LSTM(16, return_sequences=True),
    LSTM(8),
    Dense(3) # Dự đoán 3 cổ phiếu cùng lúc
])

model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.1, verbose=0)
```

```

# -----
# Tính RMSE cho các baseline
# -----
y_true = scaled_data[SEQ_LENGTH:, :] # Thực tế (close1,2,3)
y_pred_lstm = model.predict(X_test)

rmse_lstm = np.sqrt(mean_squared_error(y_test, y_pred_lstm))
print(f"RMSE LSTM: {rmse_lstm:.5f}")

# 1. Naive
y_naive = scaled_data[SEQ_LENGTH-1:-1, :]
rmse_naive = np.sqrt(mean_squared_error(y_true, y_naive))

# 2. Moving Average
window = 5
y_ma = []
for i in range(SEQ_LENGTH, len(scaled_data)):
    y_ma.append(np.mean(scaled_data[i-window:i, :], axis=0))
y_ma = np.array(y_ma)
rmse_ma = np.sqrt(mean_squared_error(y_true, y_ma))

# 3. Linear Regression
X_lr = scaled_data[SEQ_LENGTH-1:-1, :]
lr_models = [LinearRegression() for _ in range(3)]
y_lr_true = y_true.copy()
y_lr_pred = np.zeros_like(y_lr_true)
for i in range(3):
    lr_models[i].fit(X_lr, y_lr_true[:, i])
    y_lr_pred[:, i] = lr_models[i].predict(X_lr)
rmse_lr = np.sqrt(mean_squared_error(y_lr_true, y_lr_pred))

# -----
# Chọn mô hình tốt nhất
# -----
rmse_scores = {
    'Naive': rmse_naive,
    'Moving Average': rmse_ma,
    'Linear Regression': rmse_lr,
    'LSTM': rmse_lstm
}
best_model_name = min(rmse_scores, key=rmse_scores.get)

print("\n=== ĐÁNH GIÁ MÔ HÌNH ===")
for k, v in rmse_scores.items():
    print(f"{k:20s}: RMSE = {v:.5f}")
print(f"\n👉 Mô hình tốt nhất: {best_model_name} (RMSE = {rmse_scores[best_model_name]:.5f})")

# -----
# Dự báo cho tất cả cổ phiếu
# -----
last_seq = scaled_data[-SEQ_LENGTH:]
last_seq_input = np.expand_dims(last_seq, axis=0)

if best_model_name == 'LSTM':
    next_scaled = model.predict(last_seq_input)[0]
elif best_model_name == 'Linear Regression':
    next_scaled = np.array([lr_models[i].predict(scaled_data[-1,:].reshape(1,-1))[0] for i in range(3)])
elif best_model_name == 'Moving Average':
    next_scaled = np.mean(scaled_data[-5:, :], axis=0)
else: # Naive
    next_scaled = scaled_data[-1, :]

# Giải mã về giá thực tế
next_real = scaler.inverse_transform(next_scaled.reshape(1, -1))[0]
today_real = combined_data.iloc[-1].values

# Xác định xu hướng
trends = ['Tăng' if next_real[i] > today_real[i] else 'Giảm' for i in range(3)]

```

```

# -----
# Lấy ngày dự báo
# -----
last_unix = datasets[list(datasets.keys())[0]]['time'].max()
last_date = datetime.datetime.utcfromtimestamp(last_unix)
next_date = last_date + datetime.timedelta(days=1)

# -----
# Tạo bảng kết quả
# -----
results = pd.DataFrame({
    'Cổ phiếu': ['close1', 'close2', 'close3'],
    'Ngày dự báo': [next_date.strftime('%d/%m/%Y')]*3,
    'Giá hiện tại': today_real,
    'Giá dự kiến': next_real,
    'Xu hướng': trends
})

print("\n=== DỰ BÁO TỔNG HỢP ===")
display(results)

```

138/138 ————— 2s 10ms/step

RMSE LSTM: 0.00707

=== ĐÁNH GIÁ MÔ HÌNH ===

```

Naive          : RMSE = 0.00358
Moving Average : RMSE = 0.00561
Linear Regression : RMSE = 0.00357
LSTM           : RMSE = 0.00707

```

👉 Mô hình tốt nhất: Linear Regression (RMSE = 0.00357)

=== DỰ BÁO TỔNG HỢP ===

C:\Users\Admin\AppData\Local\Temp\ipykernel_21100\156457773
 ed for removal in a future version. Use timezone-aware obje
 last_date = datetime.datetime.utcfromtimestamp(last_unix)

	Cổ phiếu	Ngày dự báo	Giá hiện tại	Giá dự kiến	Xu hướng
0	close1	10/12/2024	1245.41	1245.607194	Tăng
1	close2	10/12/2024	1287.24	1286.959833	Giảm
2	close3	10/12/2024	1336.18	1336.229870	Tăng

5.

```
[13]: # Nguyễn Hoàng Tùng

import matplotlib.pyplot as plt
import seaborn as sns

# Tăng độ rõ
plt.style.use('seaborn-v0_8')
plt.rcParams['figure.figsize'] = (10,5)

# =====
# 1. Diễn biến giá theo thời gian
# =====
plt.figure(figsize=(12,6))
plt.plot(combined_data.index, combined_data['close1'], label='close1')
plt.plot(combined_data.index, combined_data['close2'], label='close2')
plt.plot(combined_data.index, combined_data['close3'], label='close3')
plt.title('1 Biểu đồ diễn biến giá 3 cổ phiếu theo thời gian')
plt.xlabel('Ngày')
plt.ylabel('Giá đóng cửa (Close)')
plt.legend()
plt.grid(True)
plt.show()

# =====
# 2. Phân phối giá cổ phiếu
# =====
plt.figure(figsize=(10,6))
sns.histplot(combined_data['close1'], kde=True, color='blue', label='close1', bins=40)
sns.histplot(combined_data['close2'], kde=True, color='orange', label='close2', bins=40)
sns.histplot(combined_data['close3'], kde=True, color='green', label='close3', bins=40)
plt.title('2 Phân phối giá cổ phiếu (Histogram)')
plt.xlabel('Giá đóng cửa')
plt.ylabel('Tần suất')
plt.legend()
plt.show()

# =====
# 3. Ma trận tương quan giữa các cổ phiếu
# =====
plt.figure(figsize=(6,5))
sns.heatmap(combined_data.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('3 Ma trận tương quan giữa các cổ phiếu')
plt.show()

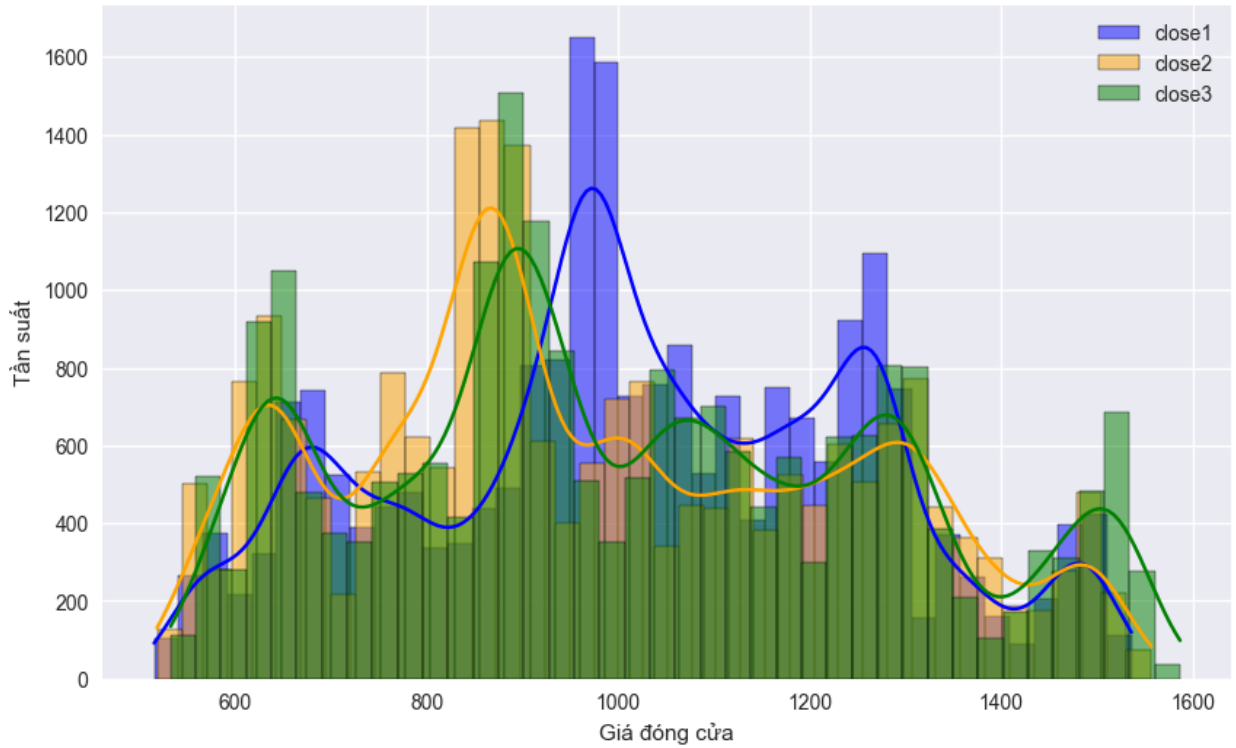
# =====
# 4. So sánh giá thực tế và giá dự báo (cho close1)
# =====
plt.figure(figsize=(12,6))
plt.plot(range(len(y_test)), scaler.inverse_transform(y_test)[:0], label='Giá thực tế (close1)', color='blue')
plt.plot(range(len(y_pred_lstm)), scaler.inverse_transform(y_pred_lstm)[:0], label='Giá dự báo LSTM', color='red', linestyle='--')
plt.title('4 So sánh giá thực tế và giá dự báo (close1)')
plt.xlabel('Thời gian')
plt.ylabel('Giá đóng cửa')
plt.legend()
plt.show()

# =====
# 5. Biểu đồ xu hướng dự báo
# =====
plt.figure(figsize=(7,5))
sns.barplot(x=results['Cổ phiếu'], y=results['Giá dự kiến'] - results['Giá hiện tại'],
            hue=results['Xu hướng'], palette={'Tăng': 'green', 'Giảm': 'red'})
plt.axhline(0, color='black', linewidth=0.8)
plt.title('5 Xu hướng biến động giá dự báo (Tăng/Giảm)')
plt.ylabel('Mức thay đổi giá dự kiến')
plt.xlabel('Cổ phiếu')
plt.show()
```

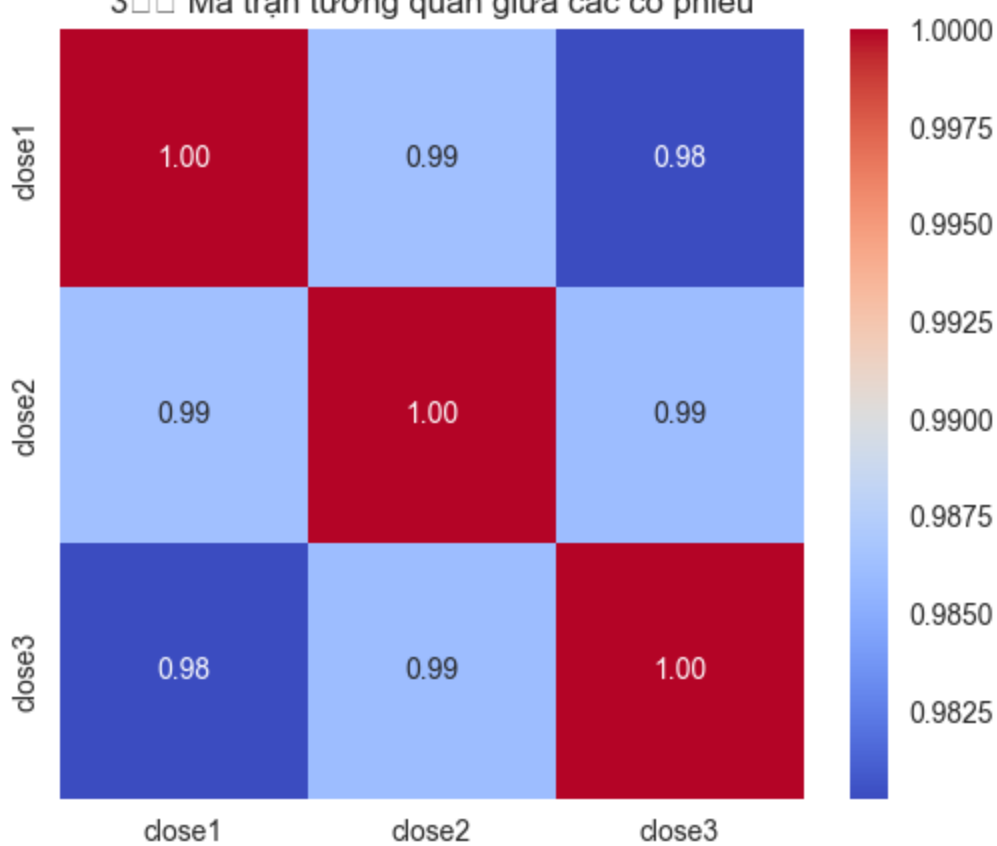
1 □ □ Biểu đồ diễn biến giá 3 cổ phiếu theo thời gian



2 □ □ Phân phối giá cổ phiếu (Histogram)



3 Ma trận tương quan giữa các cổ phiếu



4 So sánh giá thực tế và giá dự báo (close1)

